2025

Smart Scheduler for Carleton Engineering Students

Project Proposal



Smart Scheduler

Report Citation

Smart Scheduler for Carleton Engineering Students.

Smart Scheduler.

October 17, 2025.

G67 - Proff. Jerome Talim

Team Members:

Anas Ayoubi (101226450)

Hakeem Khan (101233039)

Pietro Adamvoski (101238885)

Safiullah Rattar (101245709)

Smart Scheduler
October 17, 2025

ABSTRACT

The course scheduling process for upper-year engineering students at Carleton University is inefficient and prone to error. The existing system requires students to manually select courses without contextual validation against their academic progress and provides no integrated tools for discovering and scheduling suitable electives. This project proposes the design and implementation of an intelligent course scheduler to address these deficiencies. The system will introduce two primary innovations: (1) the ability to parse a student's academic audit (in PDF or HTML format) to automatically determine completed courses and identify outstanding degree requirements, and (2) an integrated elective recommendation engine that suggests valid, schedule-compatible options. Using this data, the scheduler will generate all conflict-free timetables and rank them according to user-specified preferences. The solution will be a full-stack application developed with Next.js and TypeScript, supported by a PostgreSQL database. Project success will be measured by the accuracy of prerequisite validation, the efficiency of schedule generation, and overall usability.

TABLE OF CONTENTS

Abstract 2
1.0 Introduction 4
2.0 Background 6
3.0 Project Objective 14
3.1 Table 1: Functional Requirements 7
3.2 Table 2: Non-Function Requirements 9
3.3 Success Metrics 10
4.0 Proposed System Architecture 10
4.1 High-Level Design 10
4.2 Technology Stack 10
4.3 User Workflow 12
5.0 Methodology 14
5.1 Audit Parsing and Prerequisite Validation 14
5.2 Scheduling and Ranking Engine 15
5.3 Database Design 16
5.4 Security and Privacy Considerations 17
5.5 Testing Strategy 17
6.0 Relation to Degree Program 17
7.0 Collective Skills of the Group 19
8.0 Project Timeline and Milestones 20
8.1 Phased Timeline 20
8.2 Work Breakdown Structure 21
9.0 Risks and Mitigation Strategies 24
9.1 Technical Risks 25
9.2 Data-Related Risks 25
9.3 Project Management Risks 26
10.0 Conclusion 28
Acknowledgment 29
References 30

1.0 INTRODUCTION

For upper-year engineering students at Carleton University, constructing a valid course timetable is a complex manual task fraught with inefficiency. The university's current registration system, Carleton Central, functions primarily as an enrollment platform, leaving two significant strategic planning gaps:

1. **Manual Prerequisite and Requirement Validation:** Students must manually cross-reference their academic audit with the university calendar to select courses. This process is disconnected, time-consuming, and susceptible to errors in prerequisite checking.
2. **Lack of Elective Guidance:** The system offers no mechanism to recommend eligible technical or complementary electives. Students must consult separate university webpages to find lists of electives and then manually test each one for schedule compatibility through trial and error.

This project proposes the development of the Carleton Smart Scheduler, a web-based application designed to solve these problems by automating and optimizing the course planning process. The primary goal is to provide students with a centralized, intelligent tool that bridges the gap between their academic progress and the course registration system.
The key objectives of this project are:

1. **Automate Requirement Analysis:** To develop a robust parsing engine capable of interpreting student academic audits (in HTML or PDF format) to accurately identify completed courses and outstanding degree requirements. This provides the academic context currently missing from the manual selection process.
2. **Implement an Elective Recommendation Engine:** To build a feature that automatically identifies and suggests eligible electives that are compatible with the student's core course schedule and program requirements.
3. **Generate Conflict-Free Timetables:** To create a scheduling engine that generates all possible valid timetables based on a student's selected courses, accounting for lectures, labs, and tutorials.

4. **Implement Preference-Based Ranking:** To allow users to define scheduling preferences (time of day, schedule density) and use a scoring model to rank the generated timetables accordingly.

The scope of this project encompasses the design, implementation, and testing of a full-stack web application, including a front-end interface, a backend for business logic, and a database for course data. The system will not integrate directly with Carleton's live registration system but will provide students with the necessary Course Registration Numbers (CRNs) for manual entry. This proposal details the project's background, technical methodology, project timeline, and risk assessment.

2.0 BACKGROUND

The challenge of creating optimal university course timetables is a well-documented problem in the fields of operations research and artificial intelligence, formally known as the University Course Timetabling Problem (UCTP) [1]. The UCTP is recognized as an NP-hard problem, meaning that finding a perfect solution becomes computationally infeasible as the number of courses and constraints increases. Consequently, a variety of heuristic and algorithmic approaches have been developed to find good, practical solutions.

A common and effective method for solving such problems is to model them as a Constraint Satisfaction Problem (CSP) [2]. In a CSP model for course scheduling, the courses are variables, the available sections (lectures, labs, etc.) are the domains for those variables, and the constraints are rules such as "no two events can occupy the same time slot" and "all required components of a course must be scheduled." Algorithms such as backtracking, forward checking, and various local search heuristics are then applied to navigate the solution space and find valid timetables [2]. In practice, the state of the art can be divided into two categories of existing solutions: institutional schedulers and generic third-party tools.

1. **Institutional Schedulers:** Many universities have developed proprietary, in-house scheduling assistants. These tools are often integrated directly with the university's Student Information System (SIS). While effective, they are bespoke solutions tailored to a specific institution's data formats, academic rules, and IT infrastructure. As such, they are non-

transferable and do not offer a model that can be adapted for use at Carleton University without a significant institutional development effort.

2. **Generic Third-Party Tools:** Publicly available web applications (Coursicle, Free College Schedule Maker) provide a user-friendly interface for manually building timetables. However, their primary limitation is a lack of deep integration with institutional and student-specific data. They do not have access to a student's academic audit, cannot automatically validate prerequisites, and lack awareness of program-specific elective requirements. This forces the student back into the manual process of cross-referencing information from multiple sources.

At Carleton University, students currently rely on a combination of two disconnected tools: Carleton Central for registration and the public Carleton Course Scheduler for timetable visualization. Carleton Central serves as a transactional portal for enrollment but offers no strategic planning capabilities. The Carleton Course Scheduler, while useful for visualizing potential timetables, operates as a generic, third-party style tool. It requires students to manually input courses, cannot access a student's academic audit, and has no awareness of prerequisite completion or program-specific elective requirements. The responsibility for prerequisite validation, elective discovery, and conflict resolution thus remains entirely on the student. This fragmented and manual process is not only inefficient but also introduces a significant risk of error, which can delay graduation. This project addresses the clear gap between the capabilities of these existing tools and the specific, integrated needs of Carleton engineering students.

3.0 Project Objective

The primary objective of this project is to design, implement, and deploy a web-based intelligent course scheduling application for upper-year engineering students at Carleton University. The system aims to significantly reduce the time and complexity involved in academic planning by automating requirement validation and providing intelligent recommendations.

The specific functional and non-functional requirements that define the scope and quality of the final system are detailed below.

3.1 Table 1: Functional Requirements

ID	Requirement	Description
FR1	Academic Audit Parsing	The system shall parse an uploaded student academic audit (HTML or PDF) to extract a list of completed courses and identify program requirements.
FR2	Prerequisite Validation	The system shall validate that a student meets all prerequisite and co-requisite conditions for a selected course based on their audit or manual input.
FR3	Manual Course Entry	The system shall provide an interface for users to manually add or remove courses from their completed list and their desired schedule.
FR4	Elective Recommendation	The system shall display a list of eligible technical and complementary electives based on the user's program and remaining requirements.
FR5	Schedule Generation	The system shall generate all possible conflict-free timetables for a given set of desired courses, including all associated lab and tutorial sections.
FR6	Preference-based Ranking	The system shall rank generated schedules based on user-defined preferences (e.g., avoid morning classes, prefer compact days, keep a day free).

FR7	Interactive Timetable View	Generated schedules shall be displayed in a clear, interactive weekly calendar format.
FR8	Schedule Export	Users shall be able to export their final timetable, including Course Registration Numbers (CRNs), to a standard format (e.g., PDF, iCal).
FR9	Error and Conflict Feedback	The system shall provide clear, immediate feedback for scheduling conflicts, missing prerequisites, or other validation errors.

3.2 Table 2: Non-Functional Requirements

ID	Requirement	Description
NFR1	Performance	For a typical load of 4-6 courses, schedule generation and ranking must complete within 10 seconds. The web application must load in under 5 seconds.
NFR2	Usability	The user interface shall be intuitive, enabling a first-time user to generate a valid schedule with minimal instruction.
NFR3	Reliability	The system shall operate without crashing and handle invalid user inputs or failed audit parses gracefully.
NFR4	Compatibility	The application must be fully functional on the latest versions of major web browsers (Chrome, Firefox, Safari).
NFR5	Maintainability	The system's codebase will be modular and well-documented to facilitate future updates (e.g., updating course data for a new academic year).
NFR6	Security	All data transmission between the client and server will be encrypted using HTTPS.

3.3 Success Metrics

The success of the project will be evaluated against the following key metrics:

- **Validation Accuracy:** The prerequisite validation logic will be tested against a set of representative student scenarios. Success is defined as >95% accuracy in correctly identifying valid and invalid course selections.
- **Generation Completeness:** The scheduling engine must correctly generate all known valid timetables for a set of predefined course combinations.
- **Performance Benchmark:** The system must meet the performance target defined in NFR1 during user acceptance testing.
- **User Satisfaction:** Usability will be measured through a user testing phase with a small set of student volunteers. Success will be determined by a System Usability Scale (SUS) score of 70 or higher, indicating above-average usability.

4.0 Proposed System Architecture

This section outlines the high-level design of the Carleton Smart Scheduler, the chosen technology stack, and the expected user workflow. The architecture is designed to be modern, scalable, and maintainable.

4.1 High-Level Design

The system will be implemented as a full-stack application using Next.js. This architecture integrates the client-side user interface and the server-side business logic into a single, cohesive codebase. The design consists of three primary components:

1. **Client (Browser):** A dynamic front-end built with React and TypeScript. This component is responsible for rendering the user interface, capturing user input (such as audit file uploads, course selections, and preferences), and displaying the generated timetables. It will communicate with the backend via RESTful API calls.

2. **Backend (Next.js Server):** The server-side logic, running in a Node.js environment. This component will handle all core functionality, including:
    - Receiving and parsing academic audit files.
    - Validating course prerequisites against the database.
    - Executing the scheduling and ranking algorithms.
    - Interacting with the database via the Prisma ORM.

Database (PostgreSQL): A relational database that serves as the single source of truth for all course-related data. This includes course details, section times, prerequisite rules, and program requirements. The backend will communicate with the database exclusively through the Prisma client to ensure type-safe queries.

4.2 Technology Stack

The selection of technologies is based on ensuring type-safety, development efficiency, and robust performance. The following stack has been chosen for the project:

- **Web Framework: Next.js 14 (with App Router)**. We will use Next.js as our core framework. It is a full-stack React framework that provides an integrated solution for both frontend and backend development, simplifying our architecture and development workflow.
- **Language: TypeScript**. The entire application will be written in TypeScript. Its static typing ensures type-safety, reduces bugs, and improves code maintainability and team collaboration.
- **Database: PostgreSQL**. We will use PostgreSQL as our relational database. It is a powerful and reliable open-source system, ideal for storing the structured data of a course catalog, including prerequisites and section times.
- **ORM: Prisma**. To interact with the database, we will use Prisma. It is a modern, type-safe Object-Relational Mapper that simplifies database queries and integrates seamlessly with TypeScript, providing end-to-end type safety from the database to the client.
- **Audit Parsing Libraries**. To extract data from student audits, we will employ a dual strategy. Our **primary method** will be parsing user-provided HTML

files using a robust server-side library (Cheerio), as HTML offers a more reliable and structured format. As a **secondary objective**, we will investigate the feasibility of supporting PDF uploads by evaluating relevant libraries (e.g., pdf-parse) to handle the unstructured nature of PDF documents.

- **React UI Component Library**. The user interface will be built using a comprehensive React component library (e.g., Material-UI, Ant Design). This approach will accelerate UI development, ensure a consistent design, and provide built-in accessibility features.
- **Deployment Platform: Vercel**. The final application will be deployed on Vercel. As a cloud platform optimized for Next.js, it provides seamless continuous integration and deployment (CI/CD), allowing our team to focus on development rather than infrastructure management.

4.3 User Workflow

The system is designed for maximum flexibility, allowing users to explore schedules freely while providing intelligent guidance when requested.

1. **Course Selection:** The user begins by selecting their desired courses for the upcoming term. This is the primary and only required step to start building a schedule.
2. **Prerequisite Advising:** As courses are selected, the system will clearly display their prerequisites.
    - If an academic audit has **not** been uploaded, these prerequisites are shown for the user's information and self-assessment. The system will not block schedule generation.
    - If an academic audit **has** been uploaded, the system will automatically cross-reference the prerequisites against the user's completed courses and display a clear status icon (a green checkmark for "Met," a yellow warning for "Not Met"). This provides an immediate validation check.

3. **Enhance with Academic History:** To enable automated validation and recommendations, the user can provide their academic history at any time through one of two methods:
    - **Automated Audit Parsing:** Uploading an academic audit (HTML format preferred for reliability).
    - **Manual Entry:** Manually selecting completed courses from a master list.
4. **Elective Recommendation (If History Provided):** Once academic history is supplied, the system will feature a dedicated section suggesting eligible technical and complementary electives that are compatible with the user's core courses.
5. **Define Preferences (Optional):** The user can configure their scheduling preferences (time of day, schedule density) to influence the ranking of results.
6. **Generate and Review Schedules:** The user generates timetables. The system produces all conflict-free options. If preferences were set, the results are ranked. If an audit was provided, schedules containing courses with unmet prerequisites may be flagged but will still be displayed.

Finalize and Export: The user can browse the generated schedules, and export their final choice with the necessary Course Registration Numbers (CRNs).

5.0 Methodology

This section details the technical methods and processes that will be employed to develop the Carleton Smart Scheduler. The project will follow an iterative development model, allowing for continuous refinement of features based on testing and feedback.

5.1 Audit Parsing and Prerequisite Validation

The methodology for handling academic audits is designed to maximize reliability while exploring support for multiple formats.

- **Primary Method (HTML Parsing):** Our core strategy will focus on parsing academic audits saved as HTML files. A user will upload the file, which will be processed by a Next.js API route. On the server, a Node.js-based HTML parsing library will be used to traverse the Document Object Model (DOM) of the audit. We will identify and extract course codes ("SYSC 4907") and completion status by targeting specific HTML tags and CSS selectors. Regular expressions will be applied to standardize the extracted data. This method is prioritized due to the structured and predictable nature of HTML.
- **Secondary Investigation (PDF Parsing):** As a secondary objective, the team will investigate methods for parsing PDF audits. This is a higher-risk task due to the unstructured format of PDFs. The approach will involve using a library to extract the raw text content from the PDF file. Subsequently, a series of complex regular expressions and pattern-matching algorithms will be developed to identify course codes and grades from the text stream.
- **Validation Logic:** Once a list of completed courses is obtained (either from parsing or manual input), the prerequisite validation will be performed on the backend. When a user selects a course to add to their schedule, the system will query the PostgreSQL database via Prisma to retrieve that course's prerequisite rules. This list of required courses will then be checked against the user's list of completed courses to determine eligibility. The validation result will be sent to the client to provide immediate feedback.

5.2 Scheduling and Ranking Engine

The core of the application is the engine that generates and ranks timetables.

- **Scheduling Algorithm:** The problem of generating valid timetables will be modeled as a **Constraint Satisfaction Problem (CSP)**.
    - **Variables:** The set of courses the user wishes to take.
    - **Domains:** For each course (variable), the domain is the set of all its available sections (lectures, labs, and tutorials).
    - **Constraints:** The primary constraints are (1) **Time-Conflict Constraint**, ensuring no two selected sections overlap in time, and (2) **Completeness Constraint**, ensuring that a valid timetable includes one of each required course component (e.g., one lecture, one lab if required).
- To find all possible solutions that satisfy these constraints, we will research and implement an appropriate **constraint-solving search algorithm**. This approach will systematically explore valid combinations of course sections. The algorithm will be designed to be efficient by incorporating optimization techniques, such as pruning the search space (i.e., abandoning a partial schedule as soon as a conflict is detected), which is essential for meeting the system's performance requirements (NFR1).
- **Ranking Model:** After generating all valid, conflict-free schedules, a **heuristic scoring function** will be used for ranking. This function will evaluate each timetable against the user's selected preferences. For example, a schedule might receive positive points for having no classes on a Friday or negative points for long gaps between classes. The timetables will then be presented to the user, sorted from the highest to the lowest score.

5.3 Database Design

The system will be supported by a relational database schema in PostgreSQL.

- **Data Sourcing and Population:** The system's data will be sourced from two distinct channels:
    - **Timetable Data (Provided):** The data for course offerings, including sections, CRNs, times, and instructors for a given academic term, will be provided by the project supervisor in a structured format (e.g., JSON, CSV). This data will be ingested and used to populate the Sections table in our database.
    - **Prerequisite Data (Web Scraped):** Prerequisite rules are not included in the provided timetable data and must be sourced from the official Carleton University undergraduate calendar. We will develop a **one-time web scraping script** using a Node.js library (such as Puppeteer or Playwright) to systematically navigate the calendar website. The script will parse the HTML of each course page to extract the text describing its prerequisite and co-requisite rules. This text will be processed and structured to populate our Prerequisites relational table.
- **Schema Design:** Key entities will include:
    - Courses: Stores static information about each course (e.g., course code, title).
    - Sections: Stores the term-specific offering details provided by the supervisor.
    - Prerequisites: A relational table modeling the complex prerequisite rules obtained via web scraping.
- **Database Management:** The Prisma ORM will be used to manage the database schema through migrations and to provide a type-safe client for all database queries from the backend.

5.4 Security and Privacy Considerations

While the system requires access to sensitive academic information, the protection of student privacy is a critical consideration. Our methodology incorporates the principle of data minimization and ephemeral processing. Uploaded audit files will not be permanently stored on the server. They will be processed temporarily to extract the necessary course data and then discarded immediately. All data transmitted between the client and server will be encrypted using standard HTTPS protocols.

5.5 Testing Strategy

A multi-layered testing strategy will be implemented to ensure the system is robust, correct, and usable.

- **Unit Testing:** Individual functions, particularly for the parsing logic and scheduling algorithm, will be tested in isolation using a framework like Jest or Vitest.
- **Integration Testing:** We will test the interactions between major components, such as verifying that the frontend correctly communicates with the backend API and that the API correctly queries the database.

User Acceptance Testing (UAT): To validate usability and gather qualitative feedback, we will conduct testing sessions with a cohort of volunteer students. This will be a key part of evaluating our success metrics (Section 3.3).

6.0 Relation to Degree Program

As a team composed entirely of Software Engineering students, this project serves as a comprehensive application of the core competencies developed throughout our degree program. The project's lifecycle, from requirements analysis to deployment, directly maps to the principles of software design, development, and management central to the Software Engineering curriculum at Carleton University.

The project allows us to apply knowledge from several key domains of our program:

- **Software Design and Architecture:** Courses such as *SYSC 4120 (Software Architecture and Design)* have provided the foundation for architecting our system. We will apply principles like separation of concerns to structure the Next.js application, defining clear boundaries between the client, server, and data layers. The design of RESTful APIs for client-server communication draws directly from concepts learned in our software design projects.
- **Algorithms and Data Structures:** The development of the core scheduling engine is a direct application of knowledge from *SYSC 2100 (Algorithms and Data Structures)*. Modeling the problem as a Constraint Satisfaction Problem (CSP) and implementing an efficient search algorithm to find solutions requires a deep understanding of algorithmic complexity, graph theory, and optimization techniques.
- **Database Management:** The design and implementation of the PostgreSQL database will leverage principles from *COMP 3005 (Database Management Systems)*. This includes creating a normalized relational schema to ensure data integrity, writing efficient queries for prerequisite validation, and using an ORM like Prisma to manage data persistence.
- **Web Application Development:** The frontend development and user interface design build upon practical skills acquired in project-based courses like *SYSC 4806 (Software Engineering Lab)*. We will use these skills to build a responsive, accessible, and intuitive user interface with React, ensuring a high-quality user experience.
- **Software Engineering Process:** The overall management of the project, including requirements engineering, version control with Git, iterative development, and a structured testing strategy, reflects the software development lifecycle methodologies taught throughout the program. This project is a practical exercise in applying these processes to deliver a robust and reliable product.

In essence, this project is not just a demonstration of coding skills but a capstone exercise in applying a holistic software engineering discipline to solve a complex, real-world problem.

7.0 Collective Skills of the Group

As a team of senior Software Engineering students, we possess a complementary set of technical skills and practical experiences that cover the full software development lifecycle required for this project. Our collective expertise spans frontend and backend development, database management, and system integration, ensuring that all aspects of the Carleton Smart Scheduler can be designed, implemented, and tested to a high standard.

The specific contributions and relevant skills of each team member are outlined below:

- **Hakeem Khan (Frontend Lead & UI/UX):** Hakeem brings professional web development experience from his co-op terms at Environment Canada and CRC Ottawa. His expertise in building enterprise applications with **React, TypeScript, and JavaScript** is directly applicable to the development of the scheduler's user interface. He will lead the implementation of the frontend using a modern React component library, ensuring the application is responsive, accessible, and intuitive. His experience integrating RESTful APIs will be crucial for connecting the client-side interface to the backend.
- **Pietro Adamvoski (Full-Stack & System Integration):** Pietro has hands-on experience in full-stack development, having built applications using **JavaScript, PHP, and Spring Boot**. His understanding of the MVC (Model-View-Controller) architecture and experience implementing CRUD operations and RESTful APIs will be vital for integrating the project's various components. Pietro will focus on bridging the gap between the frontend and backend, developing API endpoints, and ensuring seamless data flow throughout the system.
- **Anas Ayoubi (Backend & Database Lead):** Anas will lead the backend development, focusing on the core logic of the application. His proficiency in **Java, Python, and SQL** makes him well-suited for designing the database schema and implementing the audit parsing logic. Anas will be responsible for developing the PostgreSQL database with Prisma, creating the data models,

and implementing the server-side validation and prerequisite-checking algorithms. His work will form the foundation of the system's "smart" capabilities.

- **Safiullah Rattar (Backend & Algorithm Development):** Safi will focus on the implementation of the core scheduling engine and the data scraping methodology. His skills in **TypeScript and Node.js** align perfectly with the chosen technology stack. He will be responsible for developing the web scraping scripts to populate our database with prerequisite data and will work closely with the team to implement the constraint-solving algorithm for schedule generation. His focus will be on ensuring the backend logic is efficient, scalable, and correctly implemented.

8.0 Project Timeline and Milestones

The project will be executed over the Fall 2025 and Winter 2026 academic terms, aligned with the key deliverables of the SYSC 4907 capstone project. The timeline is structured to ensure an iterative development process, with a functional prototype available early for testing and refinement.

8.1 Phased Timeline

Fall 2025: System Foundation and Core Functionality

The primary focus of the fall term is on project planning, architectural design, and the implementation of a minimum viable product (MVP). By the end of this term, we will have a functional application capable of generating conflict-free schedules from manually selected courses.

- **September - October:** Finalize project scope, submit proposal, design system architecture and database schema.
- **November:** Implement backend infrastructure, develop the data scraping scripts, and populate the database. Begin development of the core scheduling algorithm.

- **December:** Develop the basic frontend for course selection and timetable display. Integrate frontend with backend APIs. Prepare and submit the Progress Report.

Winter 2026: Advanced Features, Testing, and Finalization

The winter term will focus on implementing the "smart" features of the scheduler, conducting rigorous testing, and preparing the final deliverables.

- **January:** Implement the academic audit parsing feature and refine the prerequisite validation logic. Prepare for the oral presentation.
- **February:** Develop the elective recommendation engine and the preference-based ranking model. Begin drafting the final report.
- **March:** Conduct user acceptance testing (UAT) with student volunteers. Refine the UI/UX based on feedback. Finalize all features and prepare for the Poster Fair.
- **April:** Complete the final report and project video. Perform final testing, code cleanup, and deploy the production-ready application.

8.2 Work Breakdown Structure

The project is broken down into the following phases and key tasks. A distinction is made between design (engineering problem-solving) and implementation (execution) to emphasize the application of core engineering principles.

Phase 1: System Design & Planning (Complete by Nov 14, 2025)

- **T1: Requirements Engineering & System Architecture**
  - Description: Define and document system requirements, design the high-level software architecture, and select the technology stack. **(Design)**
  - Owner(s): All
  - Target Date: Oct 24, 2025
- **T2: Detailed UI/UX Prototyping**

    - Description: Create high-fidelity mockups and interactive prototypes of the user interface using a tool like Figma. This defines the user workflow and component structure. **(Design)**
    - Owner(s): Hakeem Khan
    - Target Date: Nov 7, 2025
- **T3: Database & Data Model Design**
    - Description: Design the normalized relational schema for PostgreSQL and define the data models in Prisma. **(Design)**
    - Owner(s): Anas Ayoubi
    - Target Date: Nov 7, 2025
- **T4: Core Algorithm & Parser Design**
    - Description: Research and design the constraint-solving algorithm and the HTML audit parsing strategy. **(Design)**
    - Owner(s): Safi Rattar, Pietro Adamvoski
    - Target Date: Nov 14, 2025

Phase 2: Backend Implementation & Frontend Scaffolding (Complete by Jan 17, 2026)

- **T5: Data Sourcing & Population**
    - Description: Implement the web scraping script for prerequisites and populate the database. **(Implementation)**
    - Owner(s): Safi Rattar
    - Target Date: Nov 28, 2025
- **T6: Frontend Project Setup & Component Scaffolding**
    - Description: Initialize the Next.js frontend, set up the UI library, and create placeholder components based on the T2 prototype, using mock data. **(Implementation)**
    - Owner(s): Hakeem Khan
    - Target Date: Dec 12, 2025
- **T7: Backend API & Database Implementation**
    - Description: Implement the Next.js API routes and core database logic based on the design. **(Implementation)**

  - Owner(s): Anas Ayoubi
  - Target Date: Dec 19, 2025
- **T8: Scheduling Engine & Parser Implementation**
  - Description: Implement the core scheduling algorithm and the HTML audit parser based on the design. **(Implementation)**
  - Owner(s): Safi Rattar, Pietro Adamvoski
  - Target Date: Jan 17, 2026

Phase 3: Integration & Advanced Logic (Complete by Feb 28, 2026)

- **T9: Prerequisite & Elective Logic Implementation**
  - Description: Implement the backend logic for validating prerequisites and recommending electives. **(Implementation)**
  - Owner(s): Anas Ayoubi
  - Target Date: Jan 31, 2026
- **T10: Preference-Based Ranking Model**
  - Description: Design and implement the heuristic scoring model for ranking schedules. **(Design & Implementation)**
  - Owner(s): Pietro Adamvoski
  - Target Date: Feb 14, 2026
- **T11: Full UI-Backend Integration**
  - Description: Connect the live frontend components to the backend API, replacing all mock data with real data to create a fully functional application. **(Implementation)**
  - Owner(s): Hakeem Khan, Pietro Adamvoski
  - Target Date: Feb 28, 2026

Phase 4: Testing & Refinement (Complete by Mar 28, 2026)

- **T12: System-Wide Testing**
  - Description: Develop and execute a comprehensive suite of unit, integration, and end-to-end tests for the entire system.
  - Owner(s): All
  - Target Date: Mar 14, 2026
- **T13: User Acceptance Testing (UAT) & Iteration**
  - Description: Conduct testing sessions with student volunteers, gather feedback, and implement necessary UI/UX refinements.
  - Owner(s): Hakeem Khan, Anas Ayoubi
  - Target Date: Mar 28, 2026

Phase 5: Finalization & Delivery (Complete by Apr 8, 2026)

- **T14: Final Deployment**
  - Description: Perform final code freeze, conduct regression testing, and deploy the application to the production environment on Vercel.
  - Owner(s): Safi Rattar, Pietro Adamvoski
  - Target Date: Apr 4, 2026
- **T15: Final Documentation & Deliverables**
  - Description: Complete the final report, project video, and presentation materials for submission.
  - Owner(s): All
  - Target Date: Apr 8, 2026

9.0 Risks and Mitigation Strategies

A thorough risk assessment is essential for successful project execution. We have identified potential risks in three key areas and have developed corresponding mitigation strategies to minimize their impact on the project's timeline and outcomes.

9.1 Technical Risks

- **Risk: Audit Parser Instability.** The structure of Carleton's academic audit HTML page is not guaranteed to be stable and may change without notice. Such a change could break our parser, disabling a core feature of the application. The complexity of PDF parsing presents an even higher risk of failure.
    - **Mitigation:** We will design the parser defensively, with robust error handling. If a parse fails, the system will degrade gracefully, notifying the user and seamlessly transitioning to the manual course entry mode. The application's core scheduling functionality will not depend on a successful parse. We will prioritize the more stable HTML format and treat PDF support as a secondary, higher-risk objective.
- **Risk: Algorithmic Performance.** The number of possible timetable combinations can grow exponentially with the number of selected courses and sections. A naive scheduling algorithm could lead to unacceptably long generation times, violating our performance requirements (NFR1).
    - **Mitigation:** We will benchmark the algorithm early in the development process using worst-case scenarios (e.g., 7 courses with many lab/tutorial options) to identify and address performance bottlenecks.

9.2 Data-Related Risks

- **Risk: Inaccuracy of Scraped Prerequisite Data.** The web scraping script for prerequisites relies on parsing unstructured text from the Carleton calendar. Complex or unusual prerequisite phrasings could be misinterpreted, leading to an inaccurate rule database.
    - **Mitigation:** After the initial scrape, the team will conduct a manual validation phase, cross-referencing a significant sample of the scraped rules against the official calendar to ensure accuracy. The scraper will

be designed with flexible pattern matching to handle common variations in language.

- **Risk: Stale Data.** The course catalog and timetable information change annually. If not updated, the application's data will become outdated and useless for future students.
    - **Mitigation:** The data sourcing scripts (for prerequisites and timetables) will be well-documented and designed to be re-runnable. The system's architecture will separate the application logic from the course data, allowing the database to be updated for a new academic year without requiring code changes to the core application.

9.3 Project Management Risks

- **Risk: Scope Creep.** The team may be tempted to add unplanned features (e.g., integration with the university system), which could divert resources from the core objectives and threaten deadlines.
    - **Mitigation:** We will strictly adhere to the functional requirements defined in Section 3.1. All proposed changes or additions will be subject to a formal review by the entire team to assess their impact on the schedule. The primary goal is to deliver a polished and robust version of the defined features.
- **Risk: Task Dependency Blockages.** The development of the frontend UI is dependent on the completion of the backend API. Delays in the backend could create a bottleneck for the entire project.

Mitigation: Our timeline is designed to mitigate this through parallel workstreams. The backend team will establish a clear API contract (defined data structures and endpoints) early in the process. The frontend team will then develop the UI components against this stable contract using mock data, allowing for concurrent development long before the backend logic is fully implemented.

10.0 Special Components and Facilities

This project is entirely software-based and does not require any specialized hardware, laboratory equipment, or physical facilities beyond standard development computers. All required software and services are open-source, available through free tiers, or provided by the university.

The key components required are:

- **Computing Resources:** Personal development laptops with access to a standard operating system (Windows, macOS, or Linux) are sufficient for all development tasks.
- **Software Stack:** The entire technology stack, including Node.js, Next.js, TypeScript, PostgreSQL, and Prisma, is open-source and freely available for development and deployment.
- **Data Sources:**
    - Access to the public Carleton University undergraduate calendar website is required for the prerequisite web scraping task.
    - The structured timetable data for course sections and times will be provided by the project supervisor.
- **Deployment Platform:** The project will be deployed on Vercel, which offers a comprehensive free tier that is sufficient for the development, testing, and hosting of this project.

No additional funding or special procurement is required for the successful completion of this project.

12.0 Conclusion

This project proposes the development of an intelligent course scheduler to address significant inefficiencies in the manual academic planning process for Carleton University's engineering students. By integrating academic audit parsing, prerequisite validation, and an elective recommendation engine, the system will provide a centralized, powerful tool that is currently lacking. The proposed architecture, methodology, and project plan are designed to produce a reliable, high-performance, and user-friendly application. The successful completion of this project will not only deliver a valuable tool to the student community but will also serve as a comprehensive demonstration of the software engineering principles and practices acquired throughout our degree program.

ACKNOWLEDGEMENT

We would like to acknowledge that this paper is the result of our own work and our supervisor, Prof. Jerome Talim, and we extend our gratitude to our supervisor for providing valuable feedback and guidance.

REFERENCES

[1] *I. Sbeity, M. Dbouk, and H. Kobeissi, "Combining the analytical hierarchy process and the genetic algorithm to solve the timetable problem," arXiv preprint, Sep. 2014.*

https://arxiv.org/abs/1409.2650

[2] *L. Zhang, Solving the timetabling problem using constraint satisfaction programming, MInfoSys thesis, University of Wollongong, Australia, 2005.*

https://ro.uow.edu.au/articles/thesis/Solving_the_timetabling_problem_using_constraint_satisfaction_programming/27827784

[3] *T. Schaub, "Answer set programming as a modeling language for course timetabling," Theory and Practice of Logic Programming, vol. 8, no. 5, pp. 541–564, Sep. 2008.*

https://www.cambridge.org/core/journals/theory-and-practice-of-logic-programming/article/answer-set-programming-as-a-modeling-language-for-course-timetabling/915CCB0FC4E2C1F1BE4DD71D766C793B

[4] *R. K. Thakur, N. K. Agrawal, and P. Kumar, "A practical approach to college timetable scheduling," Mathematical Modeling and Computing, vol. 11, no. 3, pp. 710–719, 2024.*

https://science.lpnu.ua/mmc/all-volumes-and-issues/volume-11-number-3-2024/practical-approach-college-timetable-scheduling

Last Page of Document